第1章　序論

1.1　プログラミング言語と言語処理系

計算機はあらかじめ決まった手順で計算を実行して答えを得る．手順のことを**プログラム** (program) とよび，プログラムを作成することをプログラミングという．計算機のプログラムは決まった書き方で記述する必要があり，**プログラミング言語** (programming language) によって定める．プログラムは明確に計算機の手順を指定する必要があるため，解釈に曖昧さのないことが望ましい．このため，プログラミング言語は，曖昧性があったり文脈に依存したりする解釈が難しい人間が使う言語（自然言語）でなく，より機械的に解釈ができる**形式言語**として定められる．

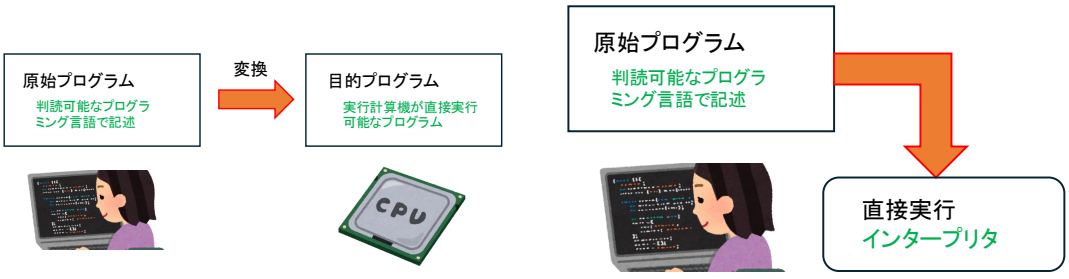
形式言語であっても自然言語にうまく対応付けることで，人間が手順をそのまま指示することができる．このような人間がそのまま読んで解釈が可能なプログラミング言語は一般に**高級言語**と呼ばれる．高級言語としては BASIC, C, Java, Python などがよく用いられる．計算の手順を適切に示すためには，対象とする計算の目的や計算機の構成に沿って記述することが自然であるため，計算のいろいろな側面に注目した高級言語が提案されている．例えば，C 言語はオペレーティングシステムを記述するために開発され，Java はオブジェクト指向という計算の捉え方に沿ってプログラムを記述する．ocaml では，プログラムは入力から出力を求める関数であるという原則に基づいて記述する．

これに対して，計算機のハードウェアを直接操作する手順は，**低級言語**と呼ばれる言語で記述され，ハードウェアごとに定められる．低級言語の中で直接，ハードウェアを操作する言語（命令列）のことを**機械語** (machine language) という．機械語は 0,1 の並びであり，計算機のアーキテクチャに応じた記述であるため，普通は人間が理解することは難しい．機械語の命令にわかりやすい名前をつけたり，メモリ領域に対して，ラベルをつけたりする機能をもたせて多少機械語のプログラムに意味を記述できるようにした**アセンブリ言語** (assembly language) がある．アセンブリ言語はハードウェア毎に存在する．アセンブリ言語のプログラムを機械語に変換するコンパイラは**アセンブラ** (assembler) と呼ばれる．

求める計算を計算機に効率的に実行させるためには，計算を実現するプログラムをプログラミング言語で記述する必要がある．人間が手順を指定するためには高級言語で記述するのが都合よい．これに対して，計算機が手順に従って計算を行うためには低級言語で操作を指定する必要がある．高級言語をなんらかの形で計算機で実行するプログラムのことを**言語処理系** (language processor) という．

計算の意味を保存したまま，高級言語から低級言語に変換すれば，効率的にプログラムを作成して実行できる．この変換を行うことを**コンパイル**（compile) といい，変換を自動的に行う計算機プログラムを**コンパイラ** (compiler) という．変換される前のプログラムを**原始プログラム** (source program)，変換の結果得られるプログラムを**目的プログラム** (object program あるいは target

program) という．人間 (プログラマー) は自然言語（主に英語）に近い形式で書かれた解釈可能な原始プログラムで手順を記述し，コンパイラで計算機ハードウェアが直接実行可能な低級言語のプログラムに変換することで，目的となるプログラムを実行することができる．コンパイラは原始プログラムを全部読み込み，目的プログラムに変換するのが普通である．



(a) コンパイラ (b) インタープリタ

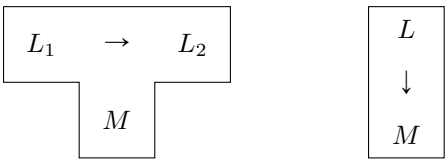
図 1.1: プログラミング言語処理系

これに対して，原始プログラムの手順としての解釈を逐次的に直接的に計算機で実行することによって原始プログラムを計算機で実行することもできる．このような実行プログラムを**インタープリタ** (interpreter) という．インタープリタは，原始プログラムの意味を直接的に実行する．インタープリタは原始プログラムを低級言語に変換することなく，直接的に解釈実行する言語処理系である．

コンパイラとインタープリタ コンパイラとインタープリタはいずれも原始プログラムを実行するための言語処理系である．コンパイラによる実行は原始プログラムを目的プログラムに変換して目的プログラムを実行する．これに対してインタープリタは原始プログラムを直接実行するようなプログラムである．インタープリタは原始プログラムを逐次的に実行するため，一般に実行は低速であるが，原始プログラムを変更すればすぐに変更が実行に反映される．これに対して，コンパイラによって変換された目的プログラムは高速に実行される．コンパイラによる実行では，原始プログラムを変更した場合には，再度原始プログラム全体を目的プログラムに変換してから実行し直す必要がある．

コンパイラによる変換においては，計算の意味を保存する必要がある．つまり，原始プログラムの計算と目的プログラムの計算は同じでなければならない．しかし，原始プログラムの計算は高級プログラミング言語で定義され，目的プログラムの計算は計算機の機械的な振る舞いで定義される．コンパイラによるプログラミングではこの**計算の意味**を保存することが必須条件である．したがって，コンパイラではプログラムが持つ意味をなんらかの形で正確に表現する必要がある．この意味表現に基づいて意味を保ちながらより効率的な変換である最適化が実現できる．コンパイラを構成するためには，プログラミング言語とプログラムの意味について正確に定式化する必要が生じる．

目的プログラムへ変換するとき，原始プログラムの計算をより効率的に実行するための**最適化** (optimization) が行われる．最適化は，プログラムの実行意味を変えることなく，より短い命令の系列でできる限りメモリを使わずに実行できるように目的プログラムを生成する．最適化は，コン



(a) コンパイラ (b) インタープリタ

図 1.2: T 図形

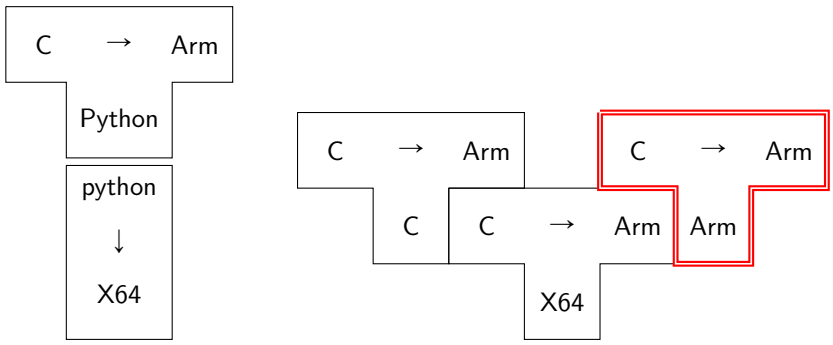
パイル技術の中核をなす概念であり，プログラミング言語を設計する観点からも重要である．

プログラムに不具合がある場合，インタープリタでは不具合は直接的に実行に反映されるが，コンパイラでは最適化の影響で不具合を特定することが難しい場合もある．プログラムの規模が大きい場合，インタープリタによる逐次的な解釈実行は現実的ではないが，小さなプログラムであれば柔軟に変更できる．Java などの言語処理系では，**JIT(Just-in-Time) コンパイラ**が導入され，インタープリタに近い柔軟な実行と目的プログラムによる効率的な実行を組み合わせる言語処理系も存在する．これに対して，原始プログラムを事前に目的プログラムに変換するコンパイラは**AOT(Ahead-Of-Time) コンパイラ**と呼ばれる．

T 図式 言語処理系の構成を示す手法として，**T 図形**が用いられることがある．図 1.2 に T 図式を示す．(a) は言語 L_1 を L_2 に変換するコンパイラが言語 M で記述されていることを示し，(b) は言語 L を M で動作するインタープリタを示す．

例 1 (図 1.3(a)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが Python で記述されているとする．X64 計算機で実行する Python インタープリタがあるとき，C 言語プログラムを X64 計算機で Arm の目的プログラムに変換することができる．

例 2 (図 1.3(b)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが C 言語で記述されているとする．X64 計算機で動作する C 言語を X64 の目的プログラムに変換するコンパイラがあるとき，X64 計算機で C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラを構成することができる．さらにこのコンパイラを最初の T 図式に適用すると，Arm 計算機上で動作するコンパイラを構成することができる．



(a) 例 1　　(b) 例 2

図 1.3: 変換過程

1.2　プログラミング言語の種類

プログラミング言語は計算の手順をどういった側面に注目して記述するかによっていくつかの種類がある．

手続き型言語: 計算の手順を順番に沿って命令の列として記述するプログラミング言語として最も標準的な記述を与える．条件分岐，ループといった制御構造の他に，再帰呼び出し，データ局所化，データ型定義などプログラムの対象に応じた命令，データ型を持つ．

関数型言語: ML(Meta Language) をはじめとする関数定義に沿ってプログラムを記述する言語である．関数は，入出力の対応を直接的に表す数学のモデルであり，プログラムの最も基本的な意味を表現する．コンパイラは関数の定義に従った計算方法を構築して目的コードに変換する．関数型言語では変換する際に，関数の型の無矛盾性について厳密にチェックすることが多い．厳密な型チェックによって矛盾した定義による不具合の発生を抑えることができる．関数の合成に基づいて，小規模な関数を合成して全体の関数を構成するという考え方に基づく．

論理型言語: 論理における証明が計算に対応するというアイデアに基づいて，ホーン節の反駁によってプログラムを計算する．Prolog が代表的な論理型言語である．証明という概念に基づいていることから，不具合が発生すること＝証明ができず正しくない，という考え方で設計される．

オブジェクト指向言語: 計算の実行主体をオブジェクトとし，オブジェクトにメッセージを送ることで計算が進行するというモデルに基づく．Java がこのアプローチをとっている．オブジェクトという概念は他の概念とは直交したモデルであるので，他の種類の言語でもオブジェクトに基づいた記述を導入している．(Python や caml など) オブジェクト指向モデルによっ

て，データのフローを局所化でき，継承に基づいた合成的な記述が可能となる．Smalltalk が本格的なオブジェクト指向言語として提案された．

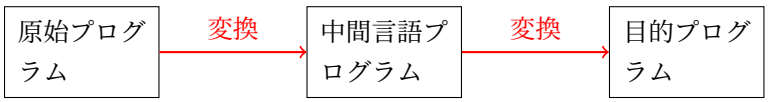
プログラミング言語は，構文 (syntax) の定義として文脈自由文法を用いることが一般的である．文脈自由文法の記法として，BNF(Backus-Naur Form) が用いられる．BNF は，文脈自由文法の生成規則を記述する．生成規則の $\rightarrow$ を ::= と記載すること，同じ左辺を持つ生成規則は | によって書くことなど，記法に違いがある．

BNF では右辺に $*$ を含めた正規表現を許す．$*$ が用いられた場合，拡張バッカス・ナウア形式 (EBNF) と呼ばれることが多い．文脈自由文法においてはメンバーシップ問題 $w \in L(G)$ は決定可能であることから，文脈自由文法で定義することにより，構文エラーは多項式時間 $\mathcal{O}(n^3)$ でチェック可能である[1]．

1.3 中間言語

コンパイラがプログラムを変換する際に一段階で変換する場合もあるが，段階を分けて変換していく場合が多い．中間的に変換される対象となる言語のことを**中間言語** (intermediate language) という．

中間言語としては，バイトコードと呼ばれるスタックマシンコードや多くのアーキテクチャに共通な特徴を持つ３番地コードなどが用いられる．中間言語はアーキテクチャに依存しない場合が多く，データフローなどプログラムそのものの性質を解析し，このレベルで最適化を行ったのち，アセンブリ言語に変換することによって，効率的に最適化を行うことができる．Java 言語処理系では，まず，バイトコードを目的プログラムとして変換し，バイトコードの実行状況に応じて，バイトコードをさらにコンパイルして，より高速に実行することが可能になる．



３番地コードは，命令コードと３つのメモリ番地を基本命令とする中間言語である．命令コードは直接機械語に対応しており，３つのメモリ番地のうち１つをデータの格納先，残り２つを計算のデータが収容されている番地を表す．３番地コードは，ハードウェアに依存しないプログラムの持つ制御フローとデータフローを表現することができる．この特徴から，計算機のアーキテクチャに依存しない基本的な最適化を適用することができる．

具体的な３番地コードについては，Java バイトコードや LLVM-IR などがある．

[1] CYK アルゴリズムと呼ばれる．

1.4　コンパイラ構成に必要な諸定義

1.4.1　語と集合演算

記号の集合をアルファベットと呼び，Σ や Γ で表す．Σ の要素を 2 個並べることによって得られる集合を $\Sigma \cdot \Sigma$ と書く．例えば，$\Sigma = \{a, b\}$ のとき $\Sigma \cdot \Sigma = \{aa, ab, ba, bb\}$ である．

一般に記号の集合 Σ, Γ に対して，**積**（あるいは連接）を $\Sigma \cdot \Gamma = \{xy \mid x \in \Sigma, y \in \Gamma\}$ と定義する．ここで，xy を x と y の**連接** (concatenation) という．記号の連接を語 (word) と呼ぶ．n 個の記号の連接 w の長さを $|w| = n$ と書く．長さ 0 の語を ε と書く．ε に対する語 w の連接は w そのものである．すなわち，$w \cdot \varepsilon = \varepsilon \cdot w = w$ である．

集合 Σ の要素を i 個連続して連接することによって得られる集合を Σ^i と書き，以下のように定められる．

$$\Sigma^i = \begin{cases} \{\varepsilon\} & i = 0 \text{ のとき} \\ \Sigma \cdot \Sigma^{i-1} & i \geqslant 1 \text{ のとき} \end{cases}$$

$\cap, \cup, \setminus$ は，2 つの集合 Σ, Γ に対して以下のようにそれぞれ積集合，和集合，差集合を構成する．

$$\Sigma \cap \Gamma = \{x \mid x \in \Sigma \text{ かつ } x \in \Gamma\} \qquad \Sigma \cup \Gamma = \{x \mid x \in \Sigma \text{ または } x \in \Gamma\}$$
$$\Sigma \setminus \Gamma = \{x \mid x \in \Sigma \text{ かつ } x \notin \Gamma\}$$

Σ の 1 個以上の連接のすべての集合を Σ^+ と書く．

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma \cup \Sigma^2 \cup \cdots \cup \Sigma^n \cup \cdots$$

$\{\varepsilon\} \cup \Sigma^+$ を Σ の**閉包** (closure) あるいは**クリーネ閉包** (Kleene closure) と呼び，Σ^* と書く．

$u, v \in \Sigma^+$ に対して，$uw = v$ を満たす $w \in \Sigma^*$ が存在するならば，u は v の**接頭語** (prefix)，$wu = v$ を満たす $w \in \Sigma^*$ が存在するならば u は v の**接尾語** (postfix) であるという．ここで，$u \cdot \varepsilon = u$ および $\varepsilon \cdot u = u$ であることから，語 u は自身の接頭語でもあり，接尾語である．

空集合は $\varnothing$ と記述する．$\varnothing \cup \Sigma = \Sigma \cup \varnothing = \Sigma$, $\varnothing \cap \Sigma = \Sigma \cap \varnothing = \varnothing$ である．連接の定義から，任意の $L \subseteq \Sigma^*$ に対して，$\varnothing \cdot L = L \cdot \varnothing = \varnothing$, $\varnothing^* = \{\varepsilon\}$ である．

1.4.2　グラフと木

グラフ (graph) は**頂点** (vertex) の集合 V と**辺** (edge) の集合 $E \subseteq V^2$ の対 (V, E) である．辺 $e = (v_1, v_2)$ は，頂点 $v_1 \in V$ から $v_2 \in V$ への矢印 $v_1 \to v_2$ で表す．v_1 は e の始点，v_2 は e の終点と呼ばれる．辺の始点と終点を区別するとき，**有向グラフ** (directed graph) という．辺 (v_1, v_2) と (v_2, v_1) を同じ辺であるとするとき，**無向グラフ** (undirected graph) という．頂点 u から出る辺の本数を u の**次数** (degree) という．$I \subseteq V$ に対して，$I^2 \cap E = \varnothing$ であるとき，すなわち I に含まれる任意の頂点の間に辺が存在しないとき，I を**独立集合** (independent set) という．

有向グラフ $G = (V, E)$ において，辺 $(u, v) \in E$ に対して，u を v の**先行頂点** (predecessor), v を u の**後続頂点** (successor) という．頂点の系列 $v_1, \cdots, v_k$ に対して，$(v_i, v_{i+1}) \in E$ が $1 \leqslant i \leqslant k-1$ に対して成り立つとき，グラフの**路** (path) という．また，頂点の系列 $v_1, \cdots, v_k$ に対して，$(v_i, v_{i+1}) \in E$

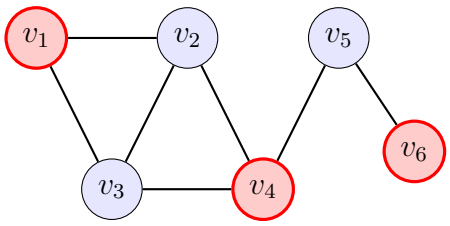
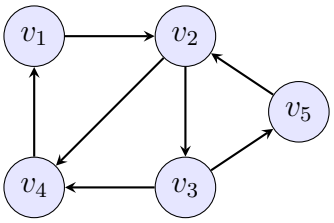


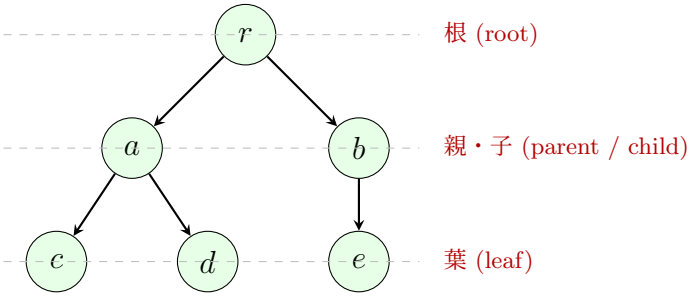
図 1.4: 無向グラフと独立集合の例

または (v_{i+1}, v_i) が $1 \leqslant i \leqslant k-1$ に対して成り立つとき，グラフの**半路** (semipath) という．任意の異なる頂点 $v_1, v_2 \in V$ に対して，v_1 から v_2 あるいは v_2 から v_1 に路が存在するとき, G は**強連結**であるといい，半路が存在するとき，G は**弱連結**であるという．

空でない頂点集合を持つ有向グラフ $G = (V, E)$ において，$v \in V$ で先行頂点を持たない頂点がただ一つ存在し，それ以外の頂点 $u \in V \backslash \{v\}$ については，ただ 1 つの先行頂点を持つとき G は**木** (tree) であるという．先行頂点を持たない頂点のことを**根** (root) という．木の条件を満たすグラフ T の根を $root(T)$ で表す．T の頂点のうち，後続頂点を持たない頂点を**葉** (leaf) という．木の条件を満たす有向グラフの頂点に対して，先行頂点を**親** (parent), 後続頂点を**子** (child) という．根は親頂点を持たず，葉は子頂点を持たない．



(a) 有向グラフの例



(b) 木の例

図 1.5: 有向グラフと木の例

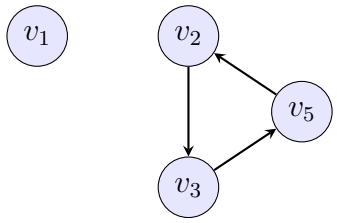
グラフ $G_s = (V_s, E_s)$ がグラフ $G = (V, E)$ の**部分グラフ**であるとは，$G_s = (V_s, E_s)$ とは，$V_s \subseteq V$, $E_s \subseteq E \cap V_s^2$ が成り立つことである．さらに，部分グラフ G_s が弱連結であるとき，G_s は，**弱連結部分グラフ**であるという．

木の条件を満たす有向グラフ T の弱結合部分グラフを T の**部分木**という．T の部分木の集合

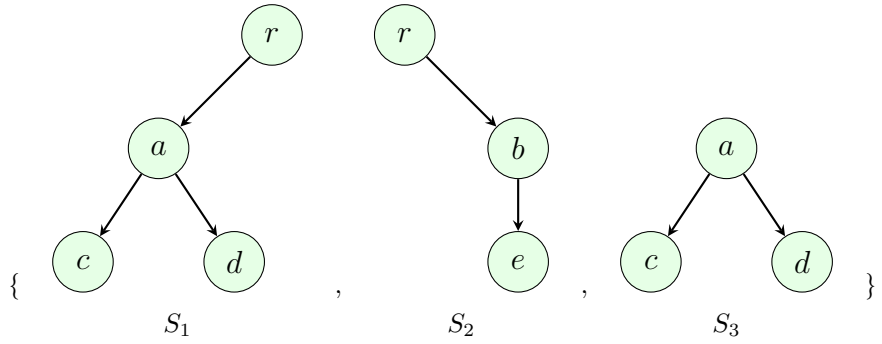
$\{S_1, \cdots, S_n\}$ の**交差グラフ (intersection graph)** は，以下のように定義される無向グラフ (V_g, E_g) である．

- $V_g = \{v_1, \cdots, v_n\}$．ここで，$v_i$ は S_i に対応する．
- $E_g = \{(v_i, v_j) \mid i \neq j, V(S_i) \cap V(S_j) \neq \varnothing\}$

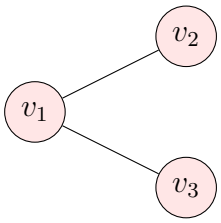
図 1.6 に，図 1.5 にあげたグラフの部分グラフ，部分木集合，および，部分木集合に対する交差グラフの例を示す．(a) については，１つの部分グラフである．部分グラフは必ずしも弱連結である必要はない．これに対して，部分木の場合は，木構造の条件を満たすために弱連結である必要がある．



(a) 部分グラフの例



(b) 部分木集合の例



(c) 交差グラフ

図 1.6: 図 1.5 の部分構造，交差グラフの例

無向グラフ (V, E) の頂点 $v \in V$ を**消去する操作**は，(V', E') というグラフを作成する操作とする．

- $V' = V \backslash \{v\}$, および
- $E' = (E \backslash E_v) \cup E_f(v)$

ここで，$E_v = \{(v_1, v_2) | v_1 = v \ or \ v_2 = v\}$, $E_f(v) = \{(v_{f1}, v_{f2}) | (v_{f1}, v), (v_{f2}, v) \in E, v_{f1} \neq v_{f2}, v_{f1} \neq v, v_{f2} \neq v\}$ とする．E_v は，削除する頂点につながっている辺であり，$E_f(v)$ は，異なる 2 辺 $e_1, e_2 \in E_v$ で v から到達する 2 つの頂点を結ぶ辺の集合である．このような辺が存在しない場合には，新たに辺を加える．すでに存在する場合はそのままとする．このようにして加えられる辺のことを **Fill-in 辺**と呼ぶ[2]．

図 1.7 において，v_1 を消去する場合は，$(v_1, v_2), (v_1, v_3)$ を消去し，(v_2, v_3) というグラフを作成する操作とする辺を追加する．v_5 を消去する場合は，すでに (v_2, v_4) に辺が存在するので，新たに辺を加えることはない．

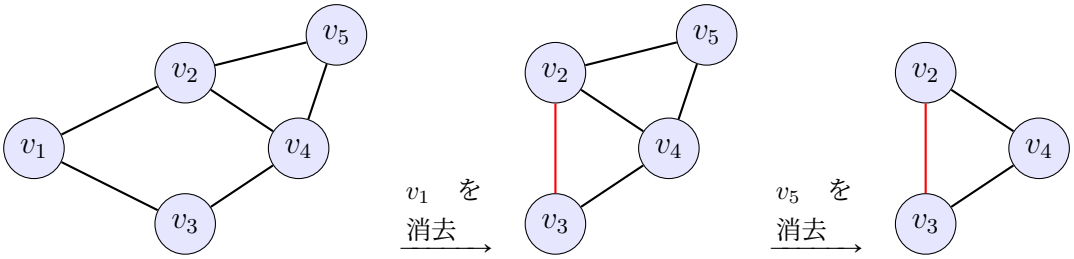


図 1.7: グラフ頂点の消去操作

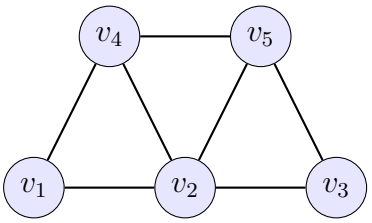
無向グラフ $G = (V, E)$ の閉路とは，相異なる頂点の系列 $(e_1, e_2, \ldots, e_k, e_1)$ $(k \geqslant 3)$ で，$(e_i, e_{i+1}) \in E$ となる辺が存在する．k を閉路の長さと呼ぶ．閉路上で隣接していない 2 頂点を結ぶ G の辺を**弦 (chord)** という．

G において $k \geqslant 4$ の任意の閉路 C に対して少なくとも 1 つの弦が存在するとき G は**弦グラフ**という．図 1.7 において，v_1 消去する前のグラフは，長さ 4 の閉路 $(v_1, v_2, v_4, v_3, v_1)$ に対して，弦が存在しないので，弦グラフではない．v_1 を消去して Fill-in 辺を追加したグラフは弦グラフである．

弦グラフ $G = (V, E)$ に対しては，以下の条件を満たす V 上の全順序が存在する．

> $v_1 < v_2 < \ldots < v_n$ のとき，$v_n, v_{n-1}, \ldots, v_1$ の順に V に消去動作を適用しても，Fill-in 辺として新たに辺が追加されない．

この順序を**完全消去順序 (Perfect Elimination Order,PEO)** という．弦グラフならば，かつそのときに限り，完全消去順序が存在することが知られている．



PEO: v_3, v_5, v_4, v_1, v_2
PEO でない: v_2, v_3, v_1, v_5, v_4

これらのグラフの概念は，主にレジスタ割り当ての際に用いる．

[2] このような Fill-in 辺を加えることをクリーク化と呼ぶこともある．